

Using LLM Function Calling for AI-Agent Interaction

Step 3: Using LLM Function Calling for Structured Execution

One of the most challenging aspects of integrating AI agents with tool execution is ensuring that the model consistently produces structured output that can be parsed correctly. Traditionally, developers would attempt to engineer prompts to make the model output a well-formatted JSON, but this approach is unstable—models can introduce variations, omit required fields, or output unstructured text that breaks parsing logic. Instead, this section introduces **function calling** as the preferred approach for structured execution. Instead of relying on the model to generate valid JSON, function calling allows us to explicitly define the tools available to the model using **OpenAI Schema**. The model then decides when and how to use these tools, ensuring consistent and predictable responses.

How Function Calling Simplifies AI-Agent Interactions

When using function calling, the model returns either:

- 1. A **function call** that includes the tool name and arguments as defined in SDX.
- 2. A **structured response** (the model decides if a tool is unnecessary).

The approach removes the need for manual prompt engineering to enforce structured output and allows the agent to focus on decision-making rather than syntax compliance.

Example: Executing Function Calls

Below is a typical way to handle function calling using OpenAI's API:

```
from typing import List, Dict, Any
from openai import OpenAI
from pydantic import BaseModel

# Define the function schema
def get_weather(location: str) -> str:
    """Returns the weather for a specific location.
    Args:
        location: The name of the city or region.
    Returns:
        A string describing the current weather conditions.
    """
    # Simulated weather data
    weather_data = {
        "New York": "Sunny, 75°F",
        "Los Angeles": "Cloudy, 68°F",
        "London": "Rainy, 55°F",
        "Tokyo": "Clear, 80°F"
    }
    return weather_data.get(location, "Unknown location")

def get_stock_price(symbol: str) -> float:
    """Returns the stock price for a given symbol.
    Args:
        symbol: The stock ticker symbol.
    Returns:
        The current stock price as a float.
    """
    # Simulated stock prices
    stock_prices = {
        "AAPL": 150.0,
        "GOOGL": 2800.0,
        "MSFT": 350.0,
        "AMZN": 3200.0
    }
    return stock_prices.get(symbol, 0.0)

# Create the OpenAI client
client = OpenAI(api_key="sk-...")

# Prepare the function schema for the model
functions = [
    {
        "name": "get_weather",
        "description": "Returns the weather for a specific location.",
        "parameters": {
            "type": "object",
            "properties": {
                "location": {
                    "type": "string",
                    "description": "The name of the city or region."
                }
            },
            "required": ["location"]
        }
    },
    {
        "name": "get_stock_price",
        "description": "Returns the stock price for a given symbol.",
        "parameters": {
            "type": "object",
            "properties": {
                "symbol": {
                    "type": "string",
                    "description": "The stock ticker symbol."
                }
            },
            "required": ["symbol"]
        }
    }
]

# Prepare the prompt and function schema
prompt = "What is the weather in New York and the stock price of AAPL?"
tools = functions

# Call the OpenAI API with function calling
response = client.chat.completions.create(
    model="gpt-4o",
    messages=[{"role": "user", "content": prompt}],
    tools=tools,
    tool_choice="auto"
)

# Parse the response
if response.choices[0].message.tool_calls:
    # The model wants to use a function
    tool_calls = response.choices[0].message.tool_calls
    for tool_call in tool_calls:
        function_name = tool_call.function.name
        arguments = tool_call.function.arguments
        # Execute the function
        result = get_weather(location=arguments) if function_name == "get_weather" else get_stock_price(symbol=arguments)
        # Append the result to the messages
        messages.append({"role": "assistant", "content": result})
else:
    # The model returned a direct response
    direct_response = response.choices[0].message.content
    messages.append({"role": "assistant", "content": direct_response})

# Final output
final_message = messages[-1].content
print(f"Final response: {final_message}")
```

Breaking Down Function Calling Step by Step

Let's break down the function calling process into five main steps:

1. Define the Tool Functions

First, we define the tools (functions) that the AI agent will use. These functions should be well-tested and handle the actual operations the AI agent can perform.

2. Create a Function Registry

We create a registry that maps function names to their corresponding Python implementations. This registry allows us to easily look up and execute the appropriate function when the model calls it.

3. Define Tool Specifications Using JSON Schema

This is where we describe our tools to the model. Each tool specification includes:

- **name**: The function name (e.g., `get_weather`).
- **description**: A brief description of what the function does.
- **parameters**: A list of input parameters with their types and descriptions.

Note how the `parameters` field uses JSON Schema to define the input structure.

4. Set Up the Agent's Instructions

We prepare the prompt and function schema for the OpenAI API call. The prompt should include the user's request, and the function schema should be passed as the `tools` parameter.

5. Prepare the Conversation Context

We maintain a list of messages representing the conversation history. This context is passed to the OpenAI API to ensure the model has access to previous interactions.

6. Make the API Call with Function Definitions

We call the OpenAI API, passing the messages and the function schema. The `tool_choice` parameter is set to `"auto"` to let the model decide when to use a tool.

7. Process the Structured Response

When using function calling, the response contains both a `function_calls` field and a `content` field. We parse the `function_calls` field to extract the tool name and arguments.

- The `function_name` is used to look up the function in the registry.
- The `arguments` are passed to the function.

If the `content` field is present, it indicates a direct response from the model.

8. Execute the Function with the Provided Arguments

Finally, we look up the appropriate function in our registry and call it with the arguments the model provided. The `try/except` block handles any errors that might occur during function execution.

Key Benefits of Function Calling APIs

• **Structured prompt engineering for structured responses**: The need to force the model to output JSON is removed.

• **Clearer model output (JSON Schema)**: The model's output is constrained to a specific schema, making it easier to parse.

• **Efficient model and tool interaction**: The model can decide whether to use a tool or provide a direct response.

• **Reusable tooling logic**: Instead of embedding tool logic directly in the prompt, it's moved to a separate `tools` parameter.

• **Generates systematically correct arguments**: The model automatically generates arguments in the correct format for the function.

Conclusion

Function calling with LLMs significantly improves the reliability of AI-agent interactions by enforcing structured responses. By defining tools with JSON Schema and letting the model decide when to use them, we build a more predictable and maintainable AI environment. In the next section, we will explore **how to register these tools dynamically using SDX** to further streamline agent development.